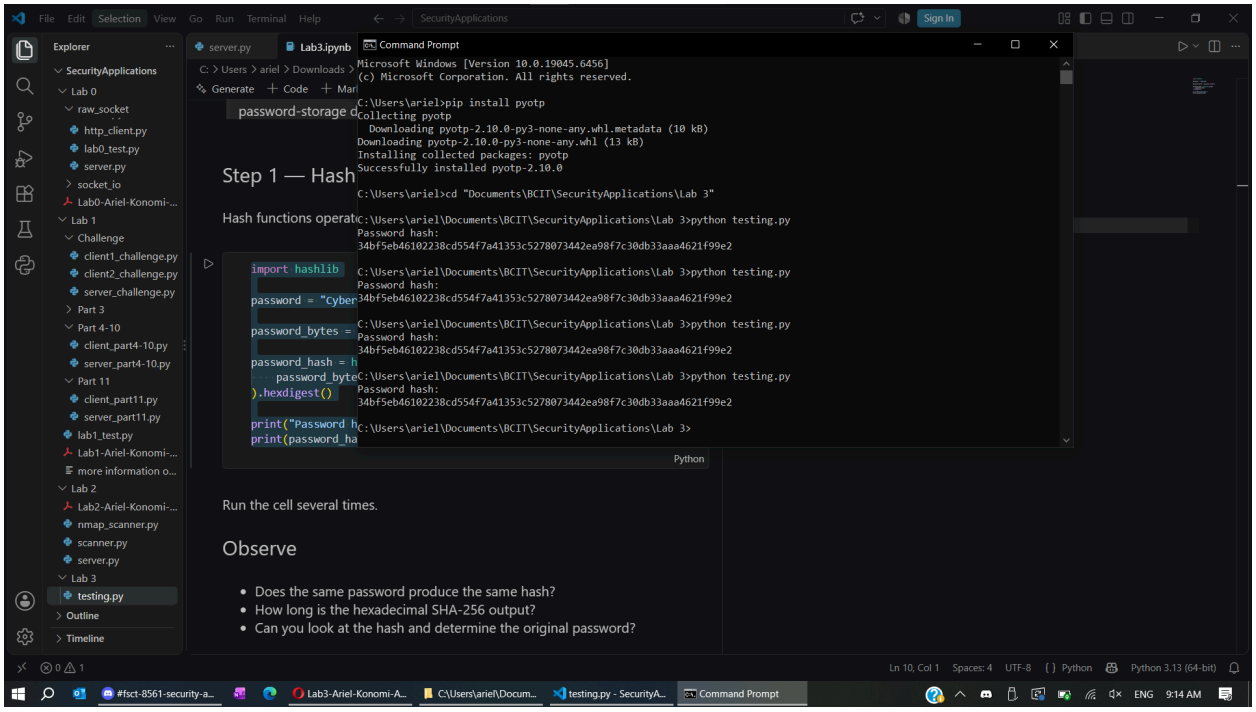
Lab 3: Building an OTP authentication server

Part 2

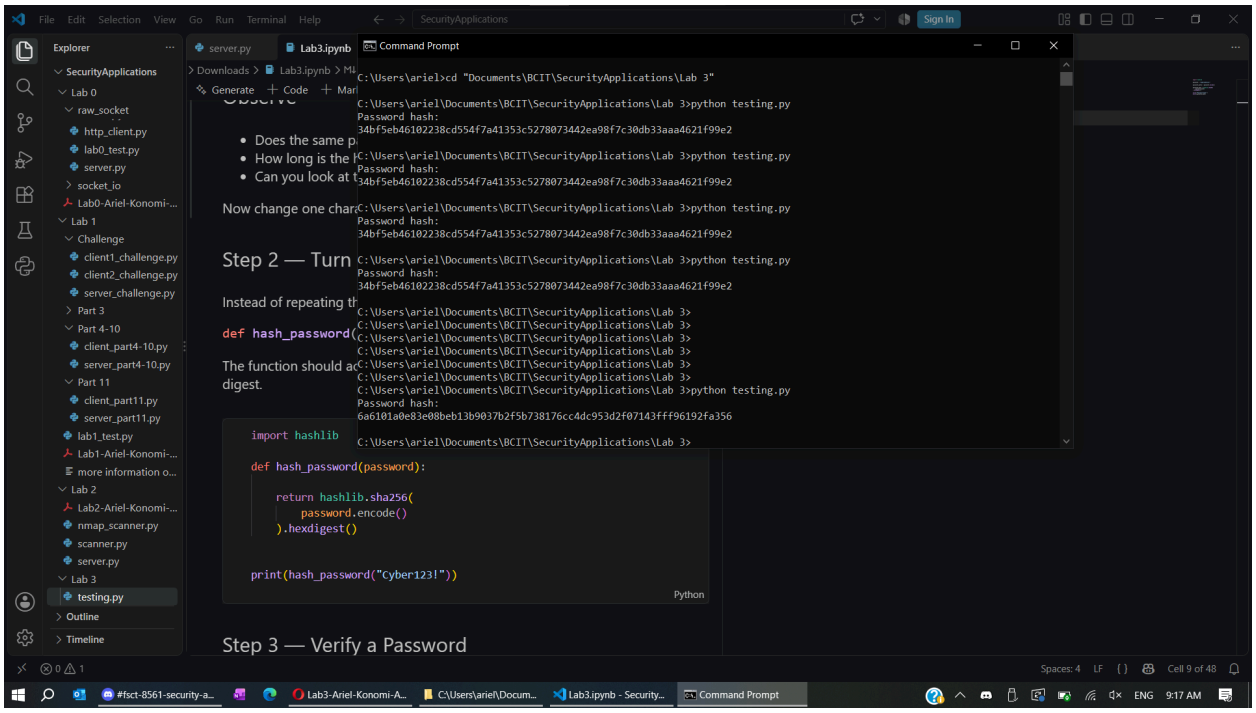
Step 1



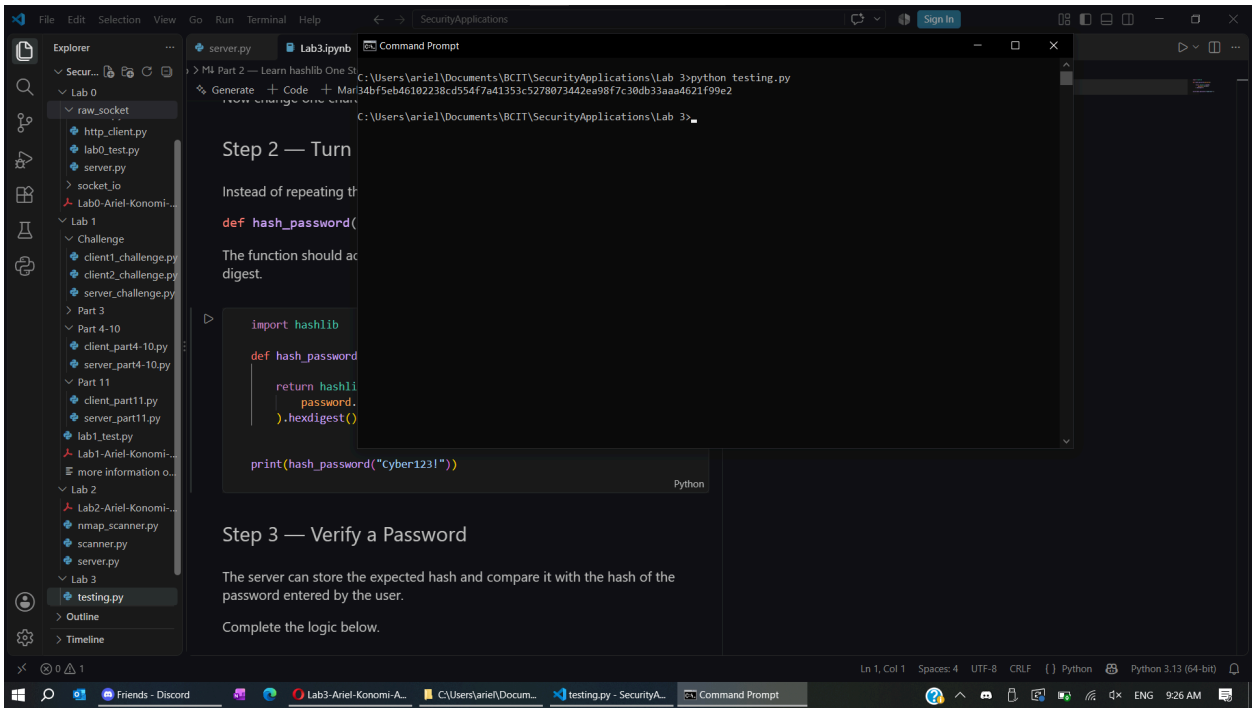
Observe

1. Does the same password produce the same hash?
    a. Yes, the same password keeps producing the same hash
2. How long is the hexadecimal SHA-256 output?
    a. 64 characters long
3. Can you look at the hash and determine the original password?
    a. No, you would not be able to determine the original password just by looking at the hash

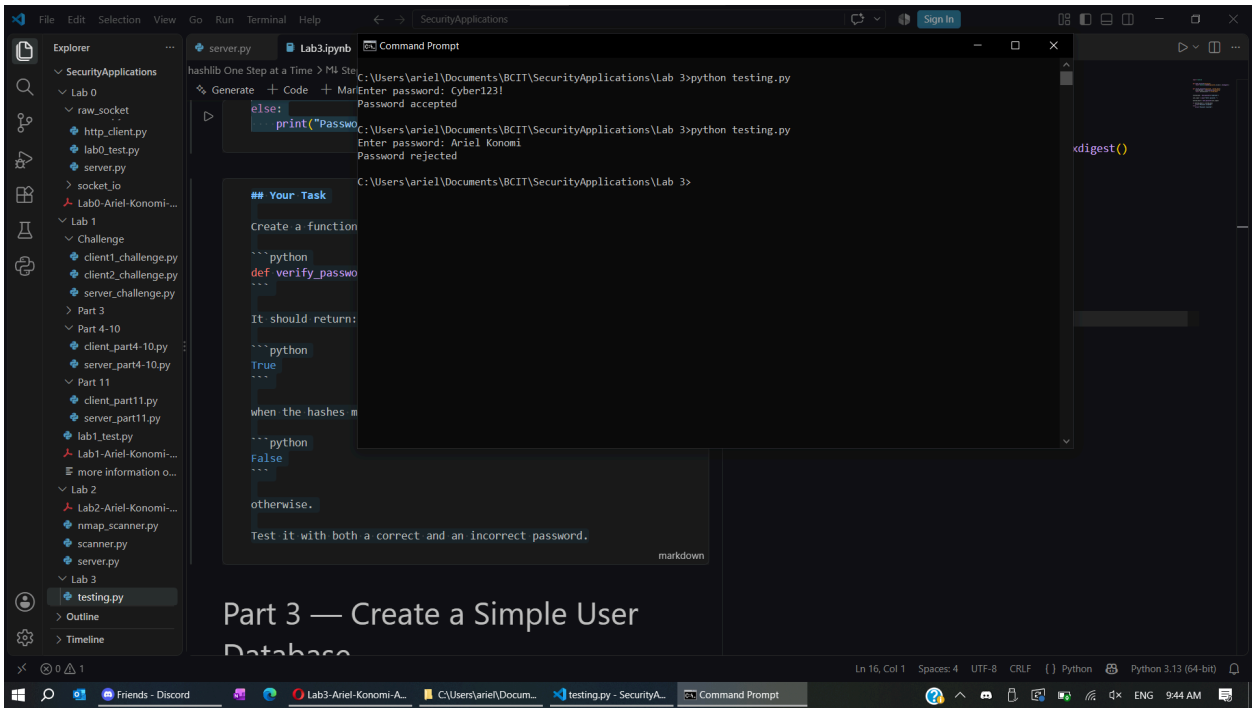
Now change one character in the password and run the code again.



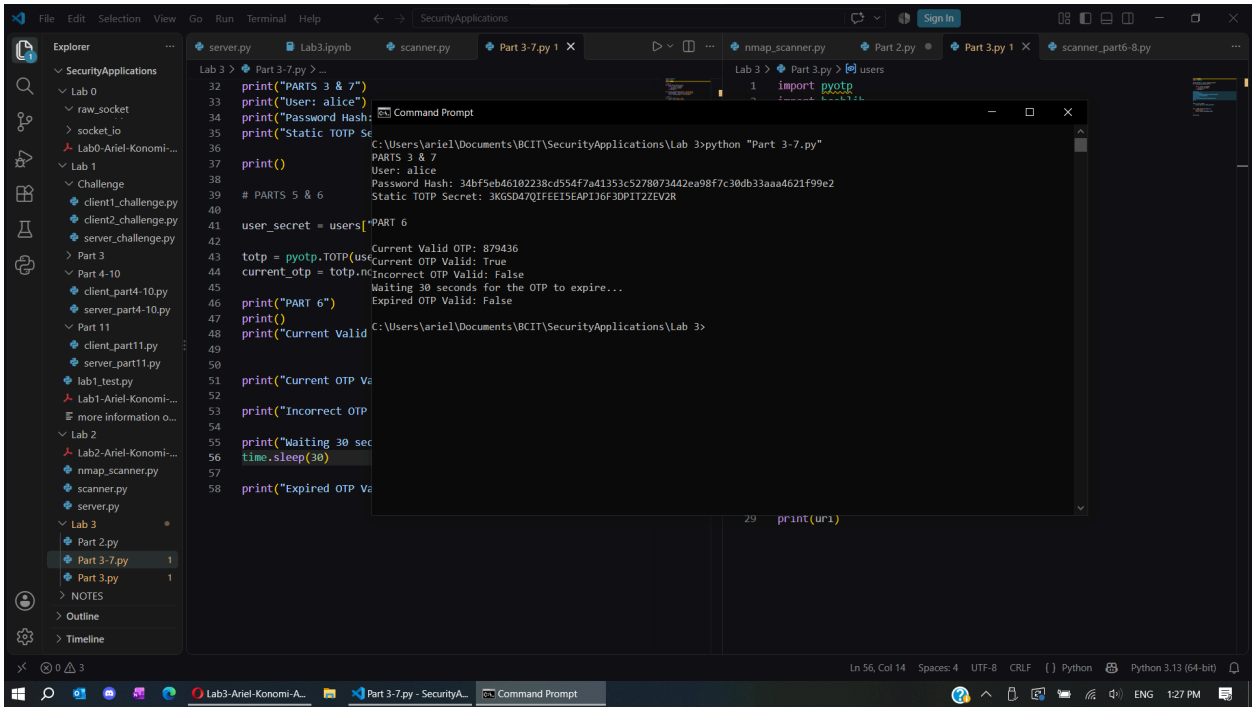
Step 2



Step 3


Command Prompt
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 3>python testing.py
Enter password: Cyber123!
Password accepted
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 3>python testing.py
Enter password: Ariel Konomi
Password rejected
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 3>
Part 3 — Create a Simple User

Part 3-7



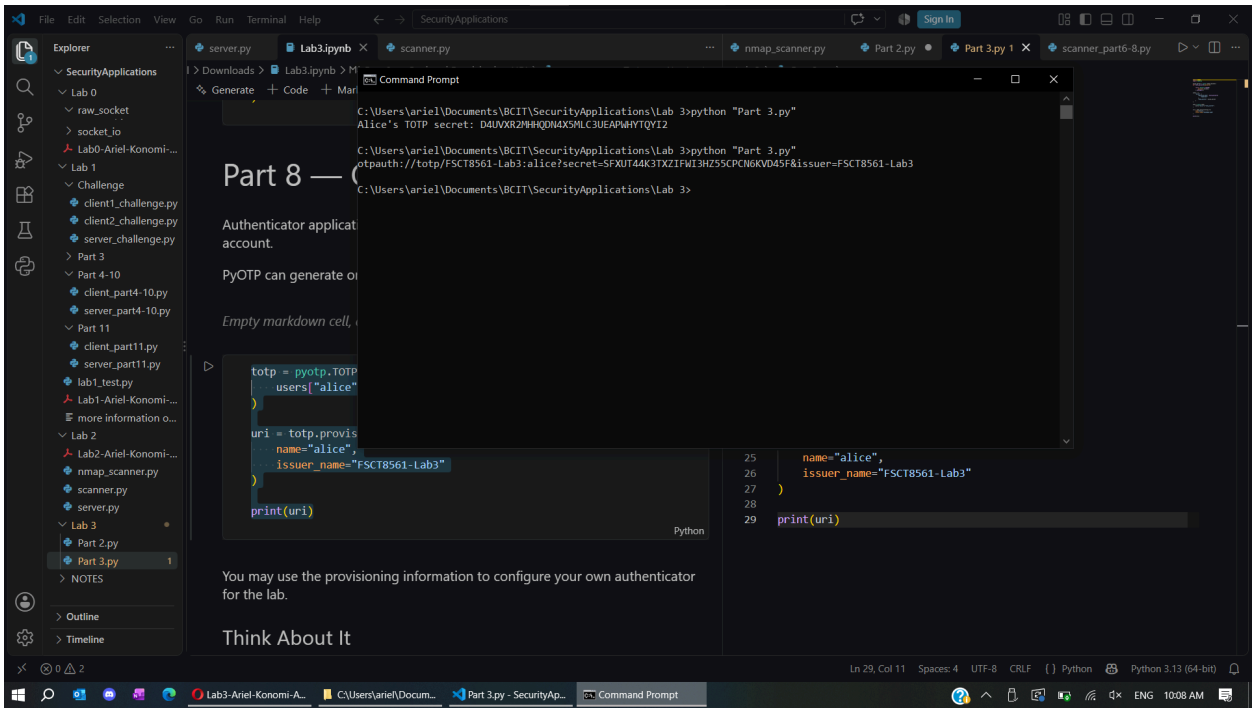
Part 5 observations

No, the pyotp.random_base(32) does not generate the same value

Part 6 Observations

1. OTP gets validated to true when tested within the 30 second window given
2. Any code that is not the code of the current time window will be false
3. If the same OTP is used after the 30 second window, it will be false

Part 8

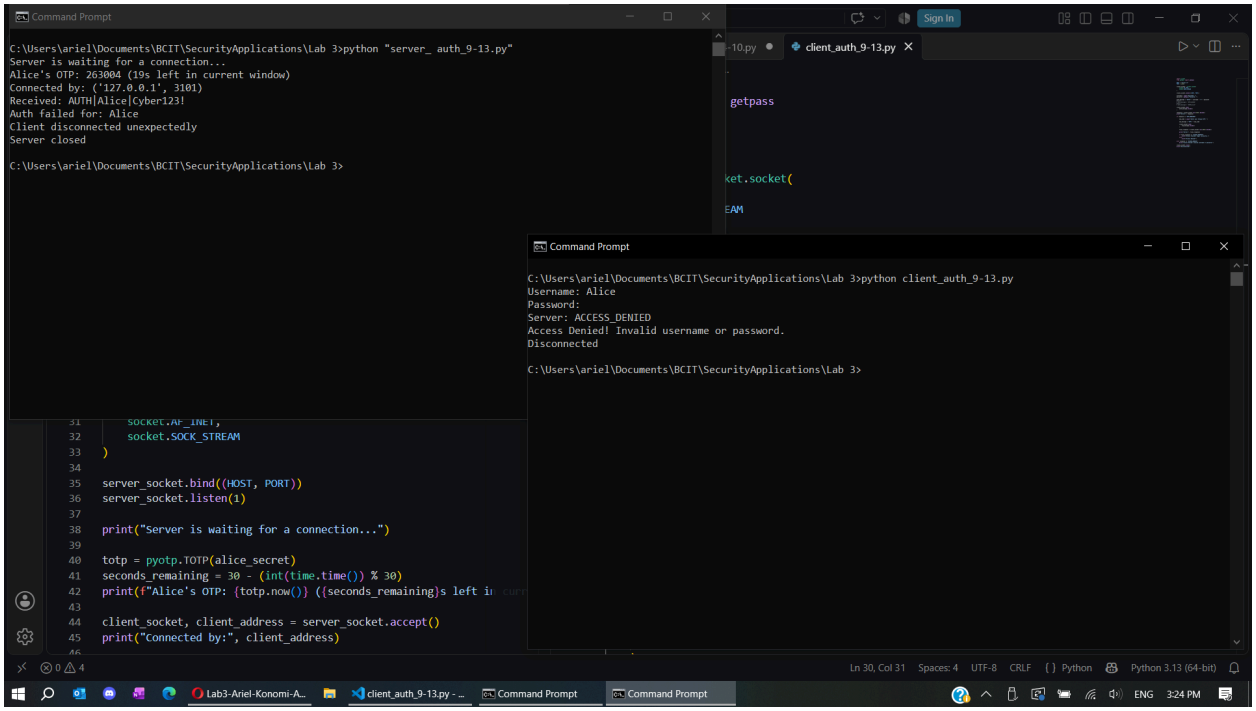


Think About It

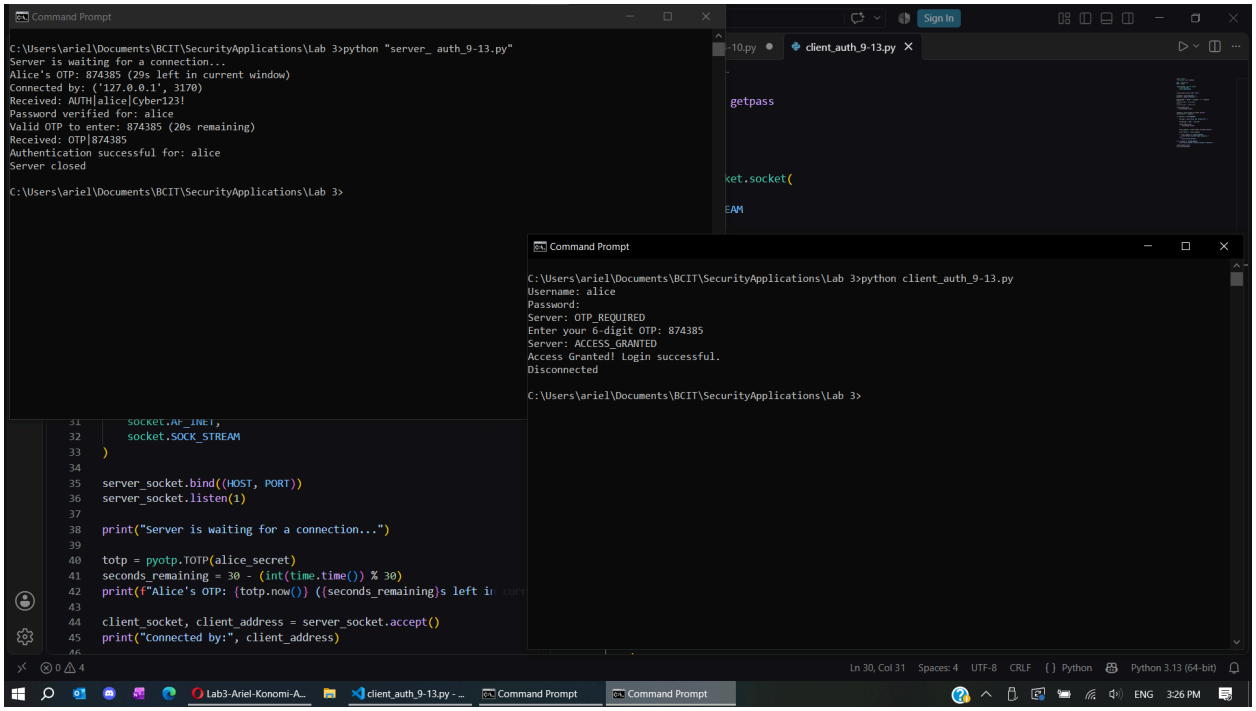
1. Why should the provisioning URI and TOTP secret **not** be exposed publicly?
    a. TOTP and URI should not be exposed publicly as they rely on being a secret. If an attacker or someone else owns a copy of the key, then anyone can authenticate as you, making the MFA a useless step

Part 9-13

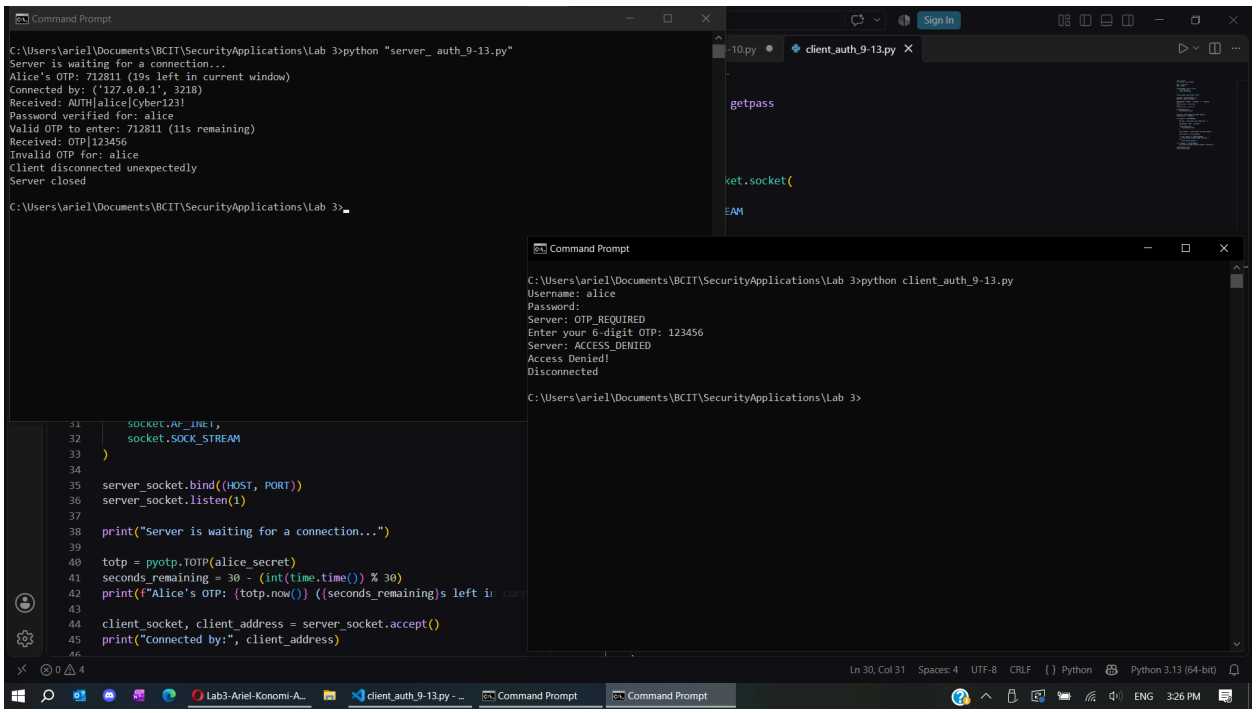
Incorrect Password



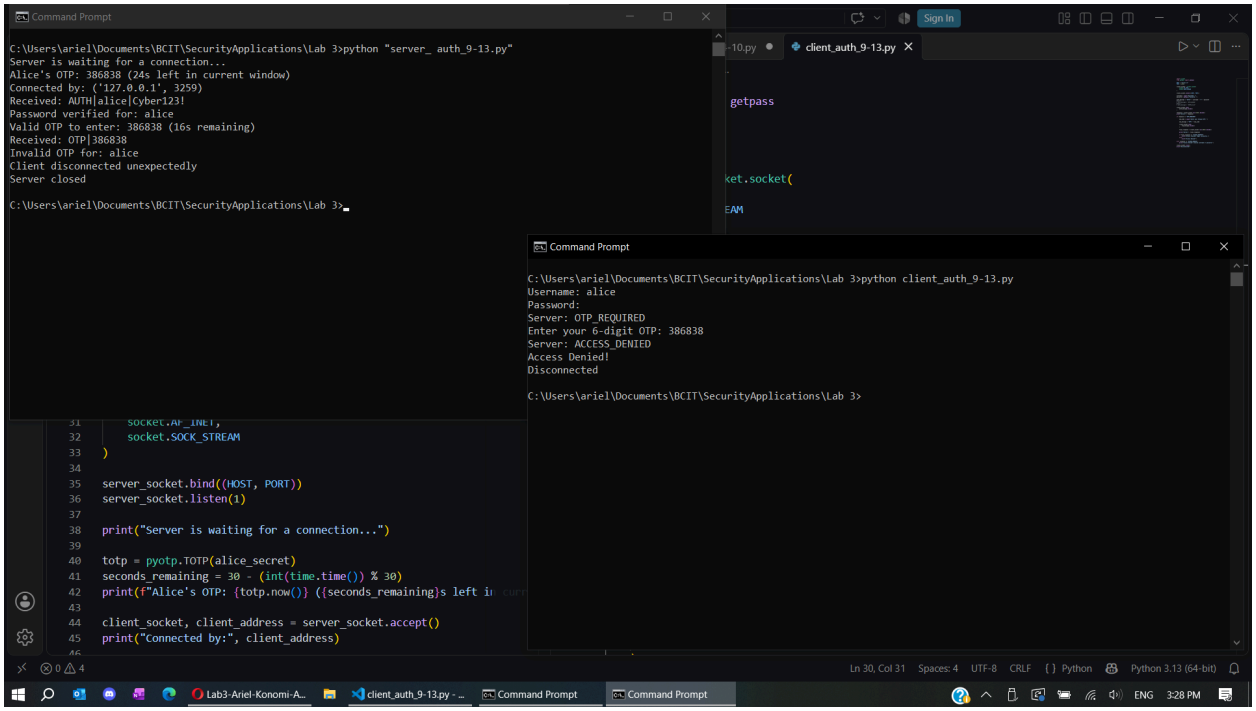
Correct Password + Correct OTP



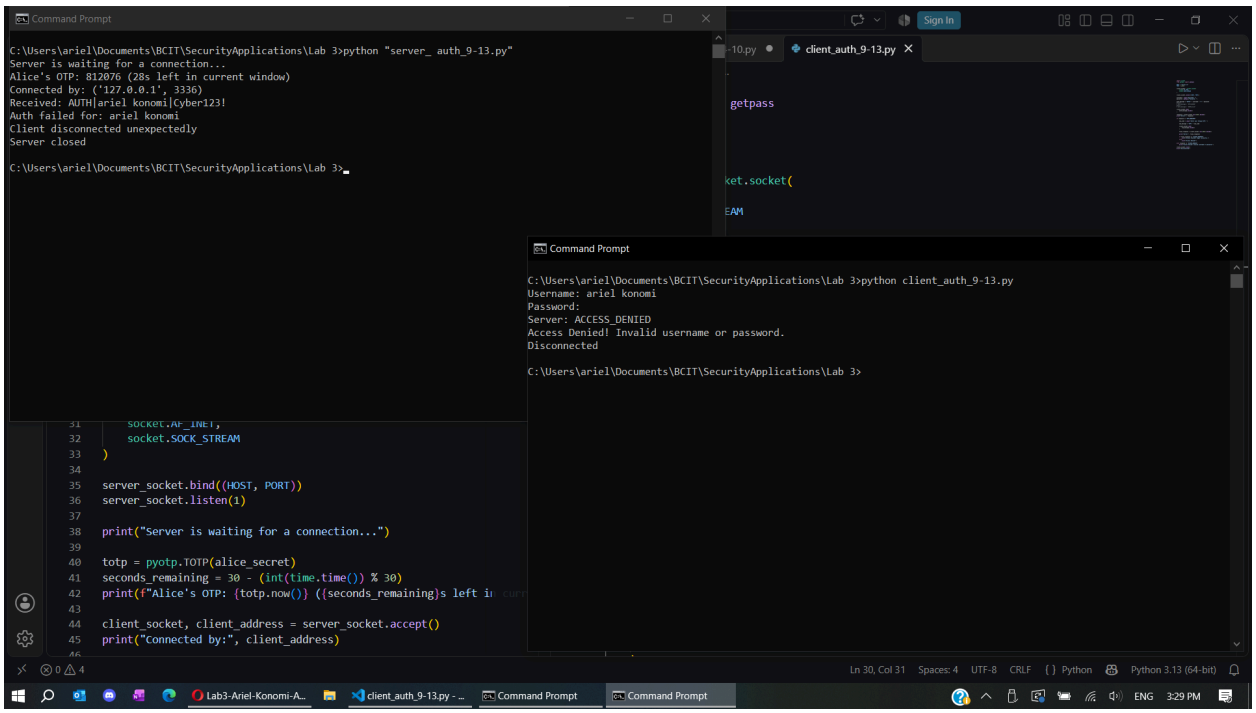
Incorrect OTP



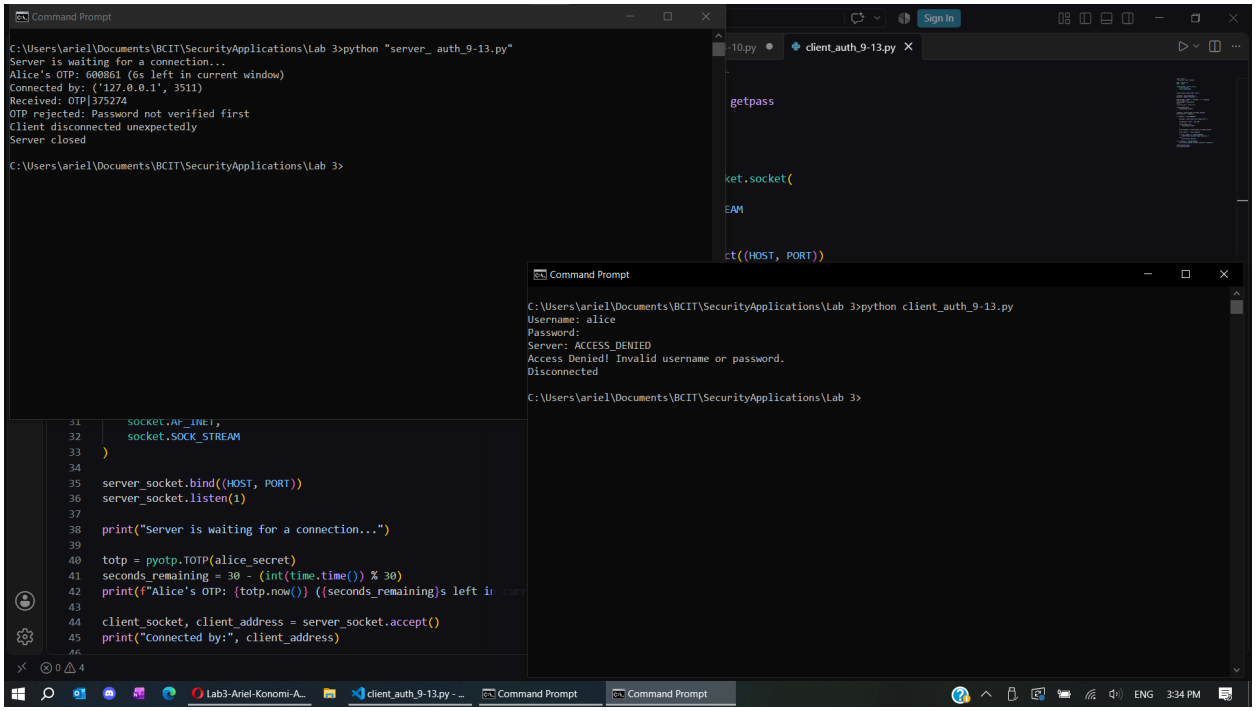
Expired OTP



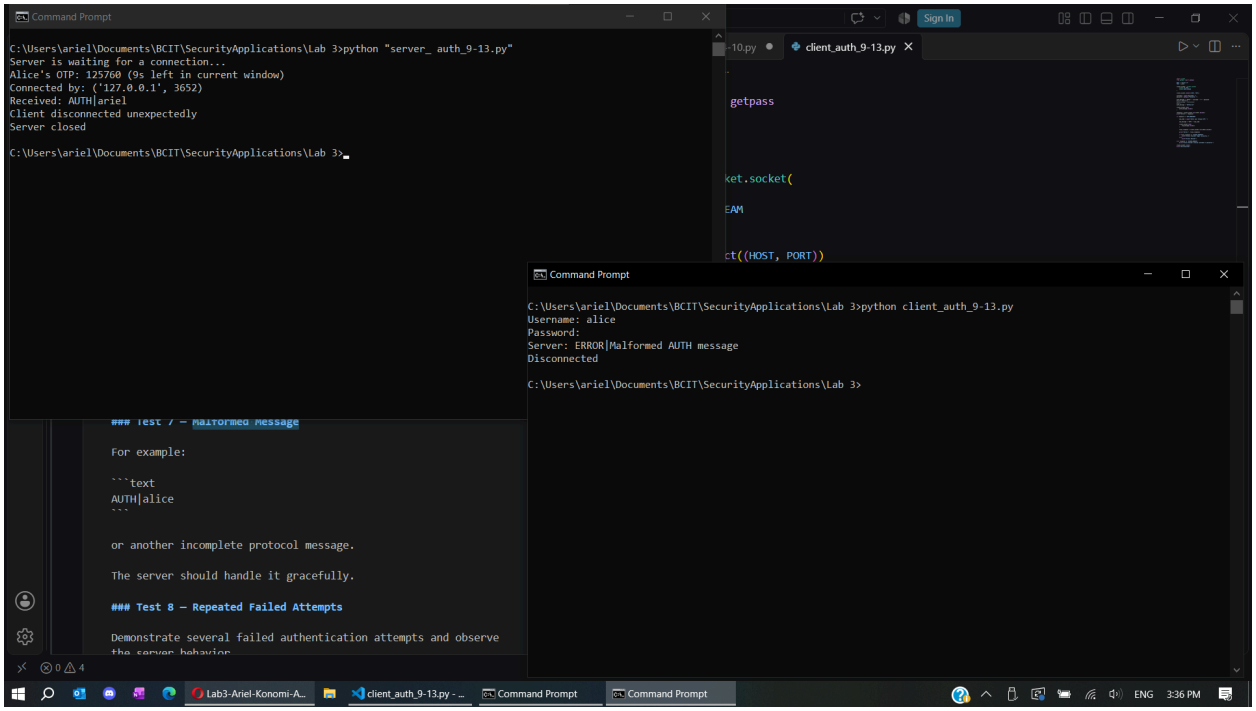
Invalid Username



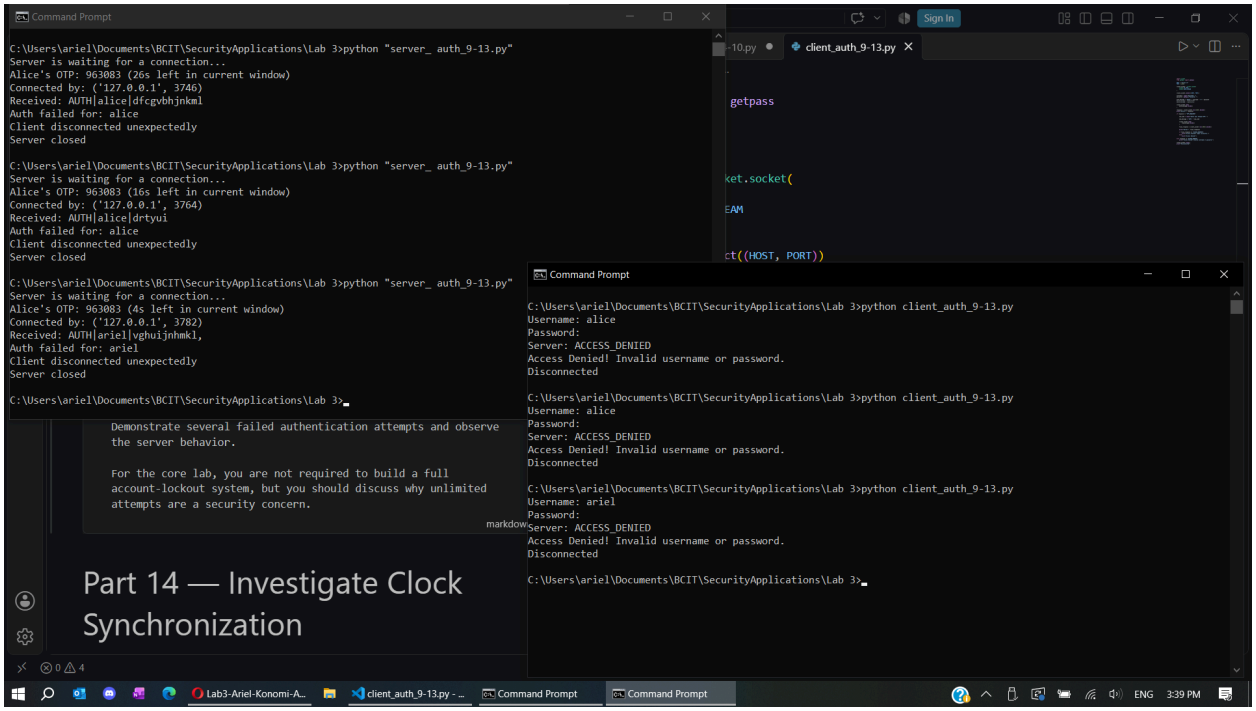
OTP Before Password Verification



Malformed Message



Repeated Failed Attempts



You should discuss why unlimited attempts are a security concern

a. Unlimited attempts are a security concern as that would give the attackers unlimited chances at trying to crack the password of the user. An automated script can be used in this case where the attackers use a dictionary attack and eventually could get in. It would also cause a DoS attack for the clients that would actually need to use the server in this case

Part 14

1. Why does TOTP depend on synchronized time?
   a. TOTP depends on synchronized time as it generates a code. This code is based on a shared secret and a specific time frame. If the time is not synchronized, then the code created, and the code being entered would not match
2. What happens when a code is submitted after its validity window?
   a. If a code is submitted after its validity window, then the code would automatically be rejected by the server
3. Why might a small amount of clock tolerance sometimes be necessary?

a. A small amount of clock tolerance could sometimes be necessary due to network delays or the time synchronization being different of different created machines. This allows a chance for the actual users to submit their code on time

4. What security trade-off could occur if the server accepts too many neighboring time windows?
    a. If the server accepts too many neighbouring time windows, the security provided by the MFA could be traded off. The attack window could be increased, as more OTPs can be accepted at the same time. This would make the brute force attacks easier to work

Part 15

1. Why should passwords not be stored in plaintext?
    a. Passwords should never be stored in plaintext. The moment a database is breached, all passwords would be exposed to the world, and if people use the same password for multiple accounts throughout different websites, even more accounts would end up being compromised
2. What is the purpose of hashing a password before storage?
    a. The purpose of hashing a password before storage would ensure that the server only stored hashed passwords. In turn, it verifies a user's identity by hashing the incoming credentials and comparing them
3. Why is plain SHA-256 alone insufficient for production password storage?
    a. SHA-265 alone would be insufficient for production password storage as it has been designed to be a hash for high speed performance. Attackers would very quickly be able to brute force these hashes
4. What additional protection does TOTP provide beyond a password?
    a. TOTP provides additional authentication by using another device like your phone. Authentication from this device would need to be provided for a user to login, even if the username and password are correct
5. How can TOTP reduce the usefulness of a captured one-time code after it expires?
    a. TOTP can reduce the usefulness of a captured one time code after it expires by generating another calculated code. Only this code would allow the user to login within a 30 second time frame
6. What happens when the user and server clocks are significantly out of sync?
    a. If the user and server clocks are significantly out of sync, then the user would not be able to login, as the calculated OTP would be different between the two
7. What happens if an attacker obtains the user's TOTP secret?

    a. If an attacker obtains the user's TOTP secret, they would easily be able to import it on their own authenticator device or application. They would be able to fully impersonate the actual user, and be able to login

8. Why should repeated failed login attempts be limited or monitored?
    a. Repeated failed login attempts should be limited or monitored to prevent brute force attacks or DoS attacks

9. Does `getpass()` protect the password while it travels across the network? Explain.
    a. 'getpass()' does not protect the password while it travels across the network. All 'getpass()' does is stops the terminal from showing the characters that the users types in to authenticate. This is to prevent other people around the user from being able to see the password the actual user provides, but the submitted password is sent via plaintext

10. What additional security would be required before deploying this authentication system in the real world?
    a. Integrating TLS
    b. Using a hashing method that would be more secure
    c. Providing temporary lockouts for accounts

Part 16

For this lab, I implemented an authentication server, which stores passwords as a hash rather than raw strings of the user's records. When the user submits their credentials, the server creates a SHA-256 digest and compares it with the stored data. By implementing pyotp, there would be two steps of authentication needed for the user to be able to log in successfully. Only after the user enters their correct username and password, an OTP would be required to be submitted by the user within a 30 second time frame. If the OTP is somehow entered before the credentials are satisfied, or if the OTP has expired, the user would not be able to continue on any further. Multiple tests were done to ensure that these steps would work as intended.

Although this authentication server works quite well, it does not mean that it does not have any flaws or vulnerabilities. The TCP sockets that are being used do not have any sort of encryption like TLS. Although 'getpass()' is used to prevent shouldersurfing, the credentials are still sent to the server as plaintext, which could easily be intercepted. SHA-256 is used for hashing, but as mentioned before, it is not the best to use in such a case as it could easily be brute-forced. The server also allows unlimited attempts of logging in. As there are no rate controls, attackers would be able to brute force and be able to find that TOTP combination needed.

There are a few fixes that I would try to implement for a real life environment. I would implement TLS to encrypt all network traffic. I would use another method of hashing, one that would be salted. I would limit the amount of attempts an account could have to log in within a certain timeframe.